

Part 1: The Phantom Bug

- Write a program that tosses 50,000,000 random (x, y) points into a 1×1 square to approximate π ($\pi \approx 4 \times \frac{\text{hits}}{\text{total}}$).
- Split the loop across 4 native Java threads.
- Have all 4 threads update a shared variable: `static long totalHits = 0;` via `totalHits++`.
- **Deliverable:** Run it 5 times. Record the value of π .
- *The Lesson:* The code runs fast, but gives $\pi \approx 1.8$ to 2.4 because threads overwrite each other's memory writes (data race).

```
PS C:\Users\toreb\OneDrive\Desktop\parallel> java -Xmx1024m -jar ms-python.debugger\ms-python.debugger\ms-python.exe 'c:\Users\toreb\.vscode\extensions\ms-python.debugger\ms-python.debugger\parallel\task1d.py'
Run 1: Pi = 3.14152 (Total Hits: 39268970)
Run 2: Pi = 3.14168 (Total Hits: 39271040)
Run 3: Pi = 3.14182 (Total Hits: 39272738)
Run 4: Pi = 3.14176 (Total Hits: 39271961)
Run 5: Pi = 3.14156 (Total Hits: 39269522)
```

Part 2: The Synchronization Trap

- Fix the race condition by adding synchronized to the increment or using `AtomicLong.incrementAndGet()`.
- **Deliverable:** Measure execution time vs. a plain, single-threaded for loop.
- *The Lesson:* The answer is now accurate (≈ 3.1415), but it runs drastically slower than a single thread because cores spend 95% of their cycles stalling on memory bus locks.

```
PS C:\Users\toreb\OneDrive\Desktop\parallel> cd 'c:\Users\toreb\.vscode\extensions\ms-python.debugger\ms-python.debugger\parallel\task2.py'
Single-threaded: Time = 5569.36 ms, Pi = 3.14178
Multi-threaded (Locked): Time = 10114.48 ms, Pi = 3.14204
PS C:\Users\toreb\OneDrive\Desktop\parallel>
```

Part 3: OpenMP-Style Reduction

- Eliminate shared-state locks. Give each thread a **private, local counter** (conceptually identical to `#pragma omp parallel for reduction(+:totalHits)`).
- Accumulate local hits inside the thread, then add the partial sums together *only once* at the very end when joining threads.
- **Deliverable:** Benchmark total execution time for $T \in \{1, 2, 4, 8, 16, 32\}$ threads

on their machine with \$100,000,000\$ iterations.

- Fill out this grid:

Threads (T)	Runtime (ms)	Speedup vs. 1 Thread (TN T1)	Efficiency (TSpeedup)
1	12412.67	1.0x	100%
2	6118.83	2.03	101.4%
4	3767.17	3.29	82.4%
8	3287.10	3.78	47.2%
16	3732.74	3.33	20.8%

Questions

1. Look at your row for 16 threads. Why didn't your 8-core CPU run twice as fast as 8 threads?

With 16 threads, the CPU has only 8 physical cores, so multiple threads have to share the same cores. This causes overhead and context switching. Therefore, 16 threads can be slower than 8 threads.

2. Why was the synchronized version in Part 2 slower than running on one single core?

The synchronized version was slower because all threads had to wait to update the shared variable. The lock creates synchronization overhead and prevents the threads from working efficiently in parallel.

Upload your solutions before the class end-time to the Git repo that you created for benchmarks. Share repo access with @sufyanism (GitHub/GitLab).